

Runbook — Sicherung und Wiederherstellung

Stand: 19. August 2026. Gilt für den Odoo-19-Stand nach dem Cutover (`masterfischer_o19` als Lager 1, `lager2_o19` als Lager 2).

Was gesichert wird und was nicht

Bestandteil	Enthalten	Ablage
Datenbank Lager 1	ja	<code>infrastructure/backups/masterfischer_o19_<datum>.dump</code>
Datenbank Lager 2	ja	<code>infrastructure/backups/lager2_o19_<datum>.dump</code>
Odoo-Filestore	ja	<code>infrastructure/backups/odoo_filestore-<datum>.tgz</code>
Rollen und Rechte des Clusters	ja	<code>infrastructure/backups/globals-<datum>.sql</code>
Prüfsummen aller vier Teile	ja	<code>infrastructure/backups/manifest-<datum>.sha256</code>
n8n-Workflows und - Zugangsdaten	nein	eigener Weg, siehe unten
<code>.env</code> mit den Secrets	nein	bewusst nicht versioniert, gehört in einen Passwortspeicher

Das Verzeichnis `infrastructure/backups/` ist in `.gitignore` ausgenommen, weil die Dumps Passwort-Hashes enthalten. Die Sicherungen gehören deshalb **nicht** in ein Cloud-Repository und **nicht** in das Abgabepaket.

Beobachtung vom 19.08.2026, die bei einer Wiederherstellung zählt: der Filestore-Ordner im Odoo-Volume heißt `masterfischer` und stammt aus der Odoo-18-Zeit. Die Anhänge der Odoo-19-Datenbanken liegen als Binärfelder in der Datenbank selbst, nicht im Filestore. Für die Bildbewertung reicht der Datenbank-Dump daher aus; der Filestore wird nur der Vollständigkeit halber mitgesichert.

Sicherung erzeugen

Docker Desktop läuft unter Windows, die WSL-Integration ist bewusst aus. Alle Aufrufe deshalb über den vollen Pfad zu `docker.exe`.

```

DOCKER="/mnt/c/Program Files/Docker/Docker/resources/bin/docker.exe"
TS=$(date +%Y%m%d)
cd "<Projektwurzel>"

# 1. Datenbanken im Container dumpen (Format "custom", damit pg_restore greift)
for db in masterfischer_o19 lager2_o19; do
    "$DOCKER" exec mobilepickingundvoiceassistant-db-1 \
        pg_dump -U odoo -Fc -f "/tmp/${db}_${TS}.dump" "$db"
    "$DOCKER" cp "mobilepickingundvoiceassistant-db-1:/tmp/${db}_${TS}.dump" \
        "infrastructure/backups/${db}_${TS}.dump"
done

# 2. Filestore
"$DOCKER" exec mobilepickingundvoiceassistant-odoo-1 \
    sh -c 'cd /var/lib/odoo && tar czf /tmp/fs.tgz filestore'
"$DOCKER" cp "mobilepickingundvoiceassistant-odoo-1:/tmp/fs.tgz" \
    "infrastructure/backups/odoo_filestore-${TS}.tgz"

# 3. Rollen des Clusters
"$DOCKER" exec mobilepickingundvoiceassistant-db-1 \
    pg_dumpall -U odoo --globals-only > "infrastructure/backups/globals-${TS}.sql"

# 4. Prüfsummen
sha256sum infrastructure/backups/*_${TS}.dump \
    infrastructure/backups/odoo_filestore-${TS}.tgz \
    infrastructure/backups/globals-${TS}.sql \
    > "infrastructure/backups/manifest-${TS}.sha256"

```

Die Prüfsumme ist kein Ritual: `docker cp` überträgt über die Windows-Grenze hinweg, und ein stillschweigend abgeschnittener Dump fällt sonst erst bei der Wiederherstellung auf.

Sicherung prüfen

```
sha256sum -c infrastructure/backups/manifest-<datum>.sha256
```

Zusätzlich lesbar machen, ohne etwas einzuspielen:

```

"$DOCKER" exec mobilepickingundvoiceassistant-db-1 \
    pg_restore --list /tmp/masterfischer_o19_<datum>.dump | head

```

Wiederherstellung

Vorher lesen: die Wiederherstellung legt die Zieldatenbank neu an. Der laufende Stand geht dabei verloren. Vor jedem Versuch zuerst eine frische Sicherung nach dem Abschnitt oben erzeugen.

```

DOCKER="/mnt/c/Program Files/Docker/Docker/resources/bin/docker.exe"
DB=masterfischer_o19
STAMP=<datum>

# 1. Stack anhalten, damit niemand mehr schreibt (Datenbank bleibt oben)
"$DOCKER" stop mobilepickingundvoiceassistant-backend-1 \
            mobilepickingundvoiceassistant-odoo-1 \
            mobilepickingundvoiceassistant-odoo-lager-2-1

# 2. Dump in den Container legen
"$DOCKER" cp "infrastructure/backups/${DB}_${STAMP}.dump" \
            "mobilepickingundvoiceassistant-db-1:/tmp/restore.dump"

# 3. Zieldatenbank neu anlegen
"$DOCKER" exec mobilepickingundvoiceassistant-db-1 \
            psql -U odoo -d postgres -c "DROP DATABASE IF EXISTS ${DB};"
"$DOCKER" exec mobilepickingundvoiceassistant-db-1 \
            psql -U odoo -d postgres -c "CREATE DATABASE ${DB} OWNER odoo;"

# 4. Einspielen
"$DOCKER" exec mobilepickingundvoiceassistant-db-1 \
            pg_restore -U odoo -d "${DB}" --no-owner --role=odoo /tmp/restore.dump

# 5. Stack wieder hoch
"$DOCKER" start mobilepickingundvoiceassistant-odoo-1 \
            mobilepickingundvoiceassistant-odoo-lager-2-1 \
            mobilepickingundvoiceassistant-backend-1

```

Danach prüfen, bevor Entwarnung gegeben wird:

1. `https://localhost/` — Anmeldung mit `lena.lager` gelingt.
2. Ein Auftrag lässt sich öffnen und eine Position bestätigen.
3. In Odoo (`http://localhost:8069`) ist dieselbe Buchung sichtbar.
4. `make test-api` läuft durch.

n8n gesondert

n8n hält seine Workflows und Zugangsdaten in der Datenbank `n8n` und verschlüsselt die Zugangsdaten mit `N8N_ENCRYPTION_KEY` aus der `.env`. Ein Datenbank-Dump ohne diesen Schlüssel ist wertlos, und der Schlüssel im selben Ordner wie der Dump hebt die Verschlüsselung auf. Deshalb:

- Dump wie oben, aber mit `n8n` als Datenbanknamen.
- `N8N_ENCRYPTION_KEY` getrennt davon im Passwortspeicher ablegen.
- Entschlüsselte Ausleitungen der Zugangsdaten gehören **nicht** in dieses Projektverzeichnis. Eine solche Datei lag bis zum 19.08.2026 unter `backup-n8n-2026-08-04/credentials-DECRYPTED.json` und liegt seitdem außerhalb des Projektbaums mit eingeschränkten Dateirechten.

Was dieses Runbook nicht abdeckt

- Keine automatische Ausführung. Es gibt keinen Zeitplan und keinen Dienst, der die Sicherung anstößt; sie wird von Hand erzeugt.
- Keine Auslagerung an einen zweiten Ort. Die Sicherungen liegen auf derselben Maschine wie der Stack.
- Kein geprüfter Wiederherstellungslauf. Die Schritte oben sind aus dem tatsächlichen Aufbau abgeleitet, aber am 19.08.2026 nicht durchgespielt worden — ein erster Testlauf gegen eine Wegwerf-Datenbank steht aus.